

```
# Tech Spec Template

**Title**: Dynamic Currency Conversion (DCC) Integration into Razorpay OmniChannel Stack
**Author/s**: <Insert Author Name(s)>
**Team/Pod**: Payments & Cross-Border
**BU**: Payments
**Published Date**: <dd/mm/yyyy>

**Reviewer Name | Reviewed Date | Status**
- Reviewer 1 | <dd/mm/yyyy> | <Status>
- Reviewer 2 | <dd/mm/yyyy> | <Status>

---

## 1. Problem Statement
Merchants and customers increasingly demand the ability to transact in their preferred currencies, especially in cross-border scenarios.

---

## 2. Introduction and Scope
This project aims to integrate DCC into Razorpay's OmniChannel stack, enabling real-time currency conversion for both online and offline transactions.

### Scope
- Enable DCC for online and offline transactions.
- Provide real-time currency conversion rates.
- Implement customer opt-in/out workflows for DCC.
- Support revenue-sharing mechanisms for merchants.
- Integrate with the Hitachi-RBL BIN Sponsorship Model.
- Update reconciliation dashboards to include DCC transactions.

### Out of Scope
- Support for non-card payment methods.
- Implementation of new merchant onboarding flows.
- Changes to existing merchant contracts.

---

## 3. Assumptions
- Third-party providers will supply real-time DCC rates with acceptable latency (<200ms SLA).
- Merchants will opt into DCC functionality explicitly.
- Existing services (`payments-cross-border`, `pg-router`, `eze-middleware`, `terminals`, `mozart`) will be extended without requiring major rewrites.
- Compliance with forex and DCC regulations will be handled by Razorpay's legal team.

---

## 4. Current Architecture
### 4.1 Legacy System
Razorpay's current architecture supports forex transactions but lacks DCC-specific capabilities, such as real-time rate fetching, currency validation, and revenue-sharing logic.

### 4.2 Intermediate Stack
The intermediate stack includes services like `payments-cross-border`, `pg-router`, and `terminals`, which handle forex transactions and merchant onboarding.

### 4.3 Target System
The target system will integrate DCC capabilities into the existing stack, leveraging the `DCC Rate Management Service` for real-time rate fetching and validation.

---

## 5. Final Approach
### 5.1 Data Flow
#### Online Transactions
1. Customer initiates checkout via Razorpay's payment gateway.
2. `pg-router` routes the transaction to `payments-cross-border` with DCC metadata.
3. `payments-cross-border` queries the `DCC Rate Management Service` for real-time rates.
4. Transaction is processed with DCC markup and revenue-sharing logic.
5. `mozart` updates the unified dashboard with DCC transaction details.
```

Offline Transactions (POS Terminals)

- 1. Customer opts in/out for DCC at the POS terminal.
- 2. `eze-middleware` routes the transaction to `payments-card-present` with DCC metadata.
- 3. `payments-cross-border` queries the `DCC Rate Management Service` for real-time rates.
- 4. Transaction is processed with DCC markup and revenue-sharing logic.
- 5. `mozart` updates the unified dashboard with DCC transaction details.

5.2 Data Model Changes

- Extend transaction metadata to include:
 - `dcc\_opt\_in` (boolean)
 - `dcc\_rate` (float)
 - `dcc\_markup` (float)
 - `merchant\_revenue\_share` (float)

5.3 Event Consumption Logic

- Modify event consumers in `payments-cross-border` and `pg-router` to handle DCC-specific attributes.
- Add new event types for DCC rate updates and transaction reconciliation.

5.4 Context Awareness / Routing

- Extend `pg-router` to route transactions based on DCC opt-in/out status.
- Implement fallback logic for transactions where DCC rates are unavailable.

5.5 External Integrations

- Integrate with third-party providers for real-time DCC rates.
- Ensure secure communication via TLS and token-based authentication.

5.6 Rollout Specific Enhancements

- Add feature flags for enabling/disabling DCC functionality at the merchant and terminal levels.
- Implement detailed logging for DCC-related events to aid debugging and compliance.

5.7 Settings / Config Requirements

- Add configurations in `terminals` for enabling/disabling DCC at the terminal level.
- Add merchant-level settings in `mozart` for DCC opt-in.

6. Dependencies

Service Name	Dependency Description	SLA	POC
DCC Rate Provider	Real-time DCC rate fetching	<200ms	<Insert POC>
payments-cross-border	Transaction processing with DCC logic	N/A	<Insert POC>
pg-router	Routing logic for DCC transactions	N/A	<Insert POC>
eze-middleware	POS transaction routing with DCC metadata	N/A	<Insert POC>
terminals	Customer opt-in/out workflows at POS terminals	N/A	<Insert POC>
mozart	Dashboard reconciliation for DCC transactions	N/A	<Insert POC>

7. Schema Changes

- Add the following fields to the `Transaction` schema:
 - `dcc\_opt\_in` (boolean)
 - `dcc\_rate` (float)
 - `dcc\_markup` (float)
 - `merchant\_revenue\_share` (float)

8. Rollout Plan

8.1 Dry Run

- Deploy the `DCC Rate Management Service` in staging and validate rate fetching.
- Test DCC workflows in staging using mock transactions.

8.2 Merchant Level Rollout

- Enable DCC functionality for select merchants via feature flags.
- Monitor transaction logs and dashboards for discrepancies.

8.3 PG Level Rollout

- Gradually enable DCC across payment gateways, starting with low-traffic gateways.
- Monitor latency and error rates closely.

9. Rollback Strategy

- Disable DCC functionality via feature flags.
- Revert schema changes using backup scripts.
- Rollback service deployments to the previous stable version.

10. Traffic Estimates

- Online transactions: ~10,000 DCC-enabled transactions/day.
- Offline transactions: ~5,000 DCC-enabled transactions/day.
- Expected peak traffic: ~500 transactions/min during high-volume periods.

11. System Stability Plan

- Implement caching in the `DCC Rate Management Service` to reduce latency.
- Add retries with exponential backoff for rate fetching.
- Monitor system metrics (CPU, memory, latency) and set alerts for anomalies.

12. Open Questions

- What is the SLA for third-party DCC rate providers?
- Are there additional compliance requirements for specific regions?
- Should merchants be able to customize DCC markup percentages?

13. Migration Experience

- Provide detailed documentation for merchants to enable DCC.
- Offer training sessions for merchants using POS terminals.
- Ensure smooth migration by testing workflows in staging environments.

14. Future Enhancements

- Support for non-card payment methods (e.g., UPI, wallets).
- Implement AI-based rate prediction for improved accuracy.
- Add support for merchant-specific DCC markup configurations.

15. Pre and Post Migration Comparison

- **Pre-Migration**: No DCC functionality; transactions processed in base currency.
- **Post-Migration**: DCC-enabled transactions with real-time rates, customer opt-in/out workflows, and revenue-sharing mechanisms.

Document Link: [To be added once uploaded]

Slack Thread for Review: [Insert thread link]